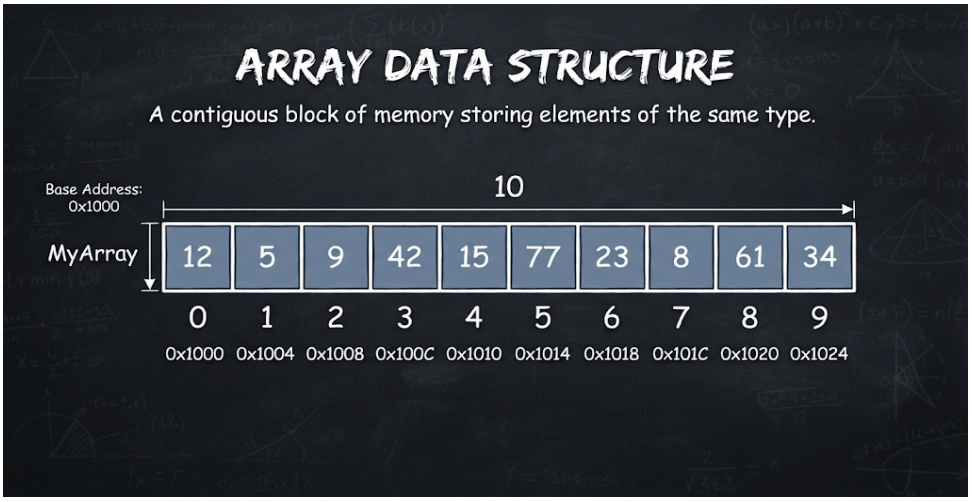
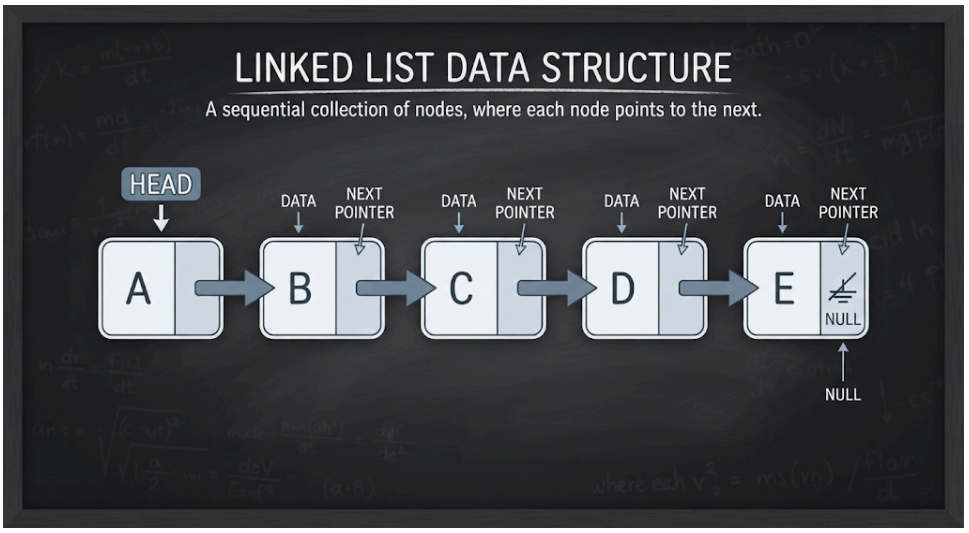
Linear Data Structures:

A **linear data structure** is a foundational way of organizing and storing data where elements are arranged sequentially or linearly. In this structure, every element is connected to its previous and next element, meaning the data can be traversed in a single sequential pass.

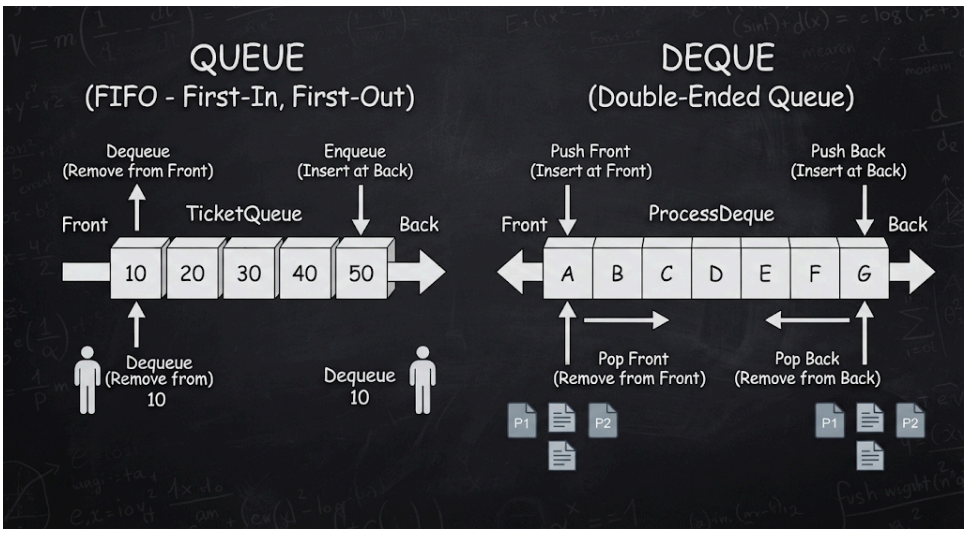
Examples:



Arrays: A contiguous block of memory storing elements sequentially where items are accessed instantly via indices.



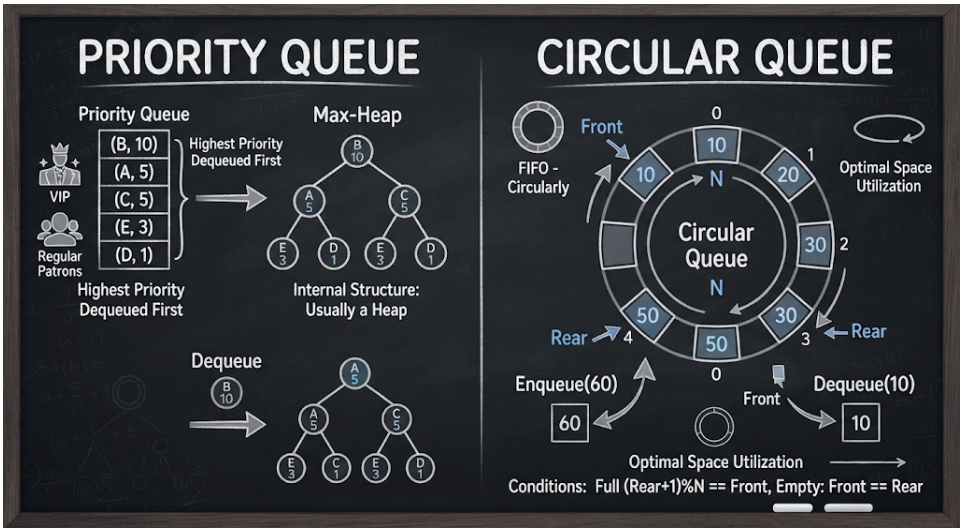
Linked Lists: A chain of nodes where each node contains data and a pointer directing you to the next sequential node.



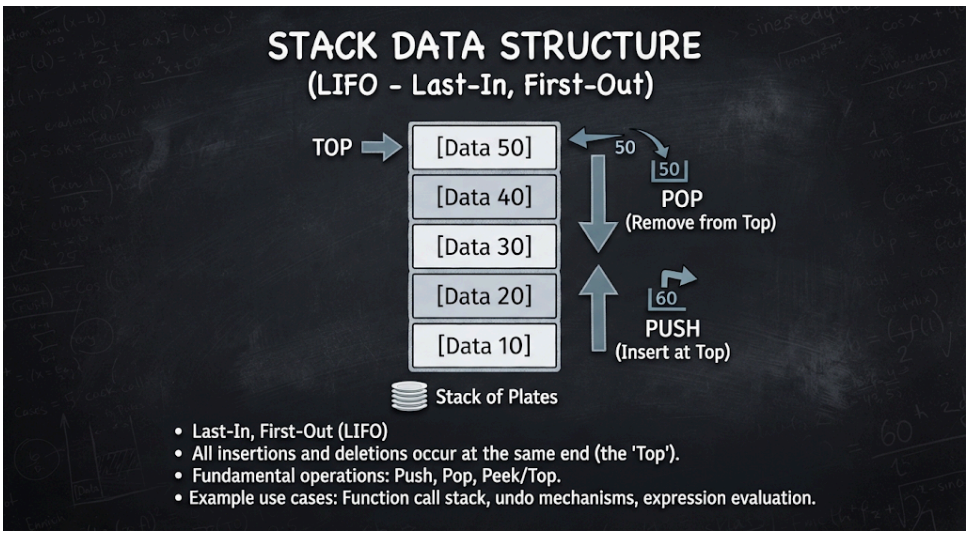
Queues (and Deques): First-In, First-Out (FIFO) structures where elements enter from the back and leave from the front.

Enqueue: Inserts a new element at the back (rear) of the queue.
Dequeue: Removes and returns the element from the front of the queue.
Peek / Front: Views the data at the front of the queue without removing it.
IsEmpty / IsFull: Helper operations to check if the queue is empty or has reached capacity.

The Issue: When elements are dequeued from the front of a static array, empty indices are left behind. If you want to reuse that vacant front space or keep the queue aligned, all remaining elements must be shifted one position to the left, which takes **O(N) time**.



The Solution: This is resolved by implementing a **Circular Queue**, which wraps the front and rear pointers around the array using modulo arithmetic to ensure optimal space utilization without costly shifting.

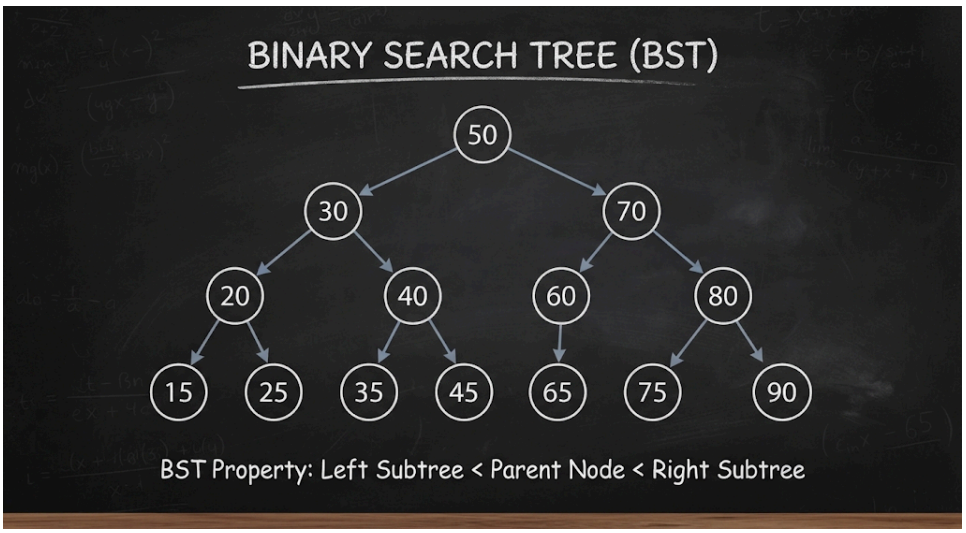


Stacks: A Last-In, First-Out (LIFO) structure where insertions and deletions happen exclusively at one end called the "top".

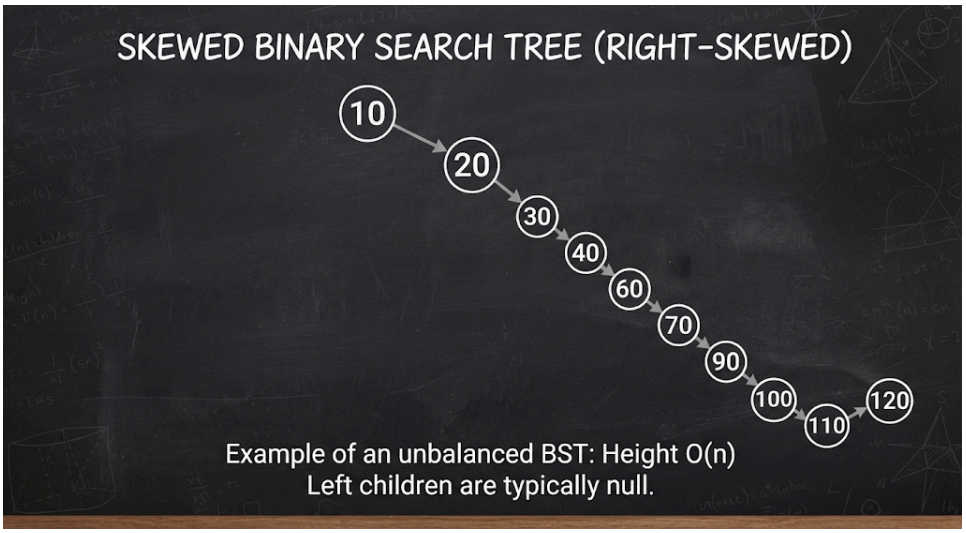
Push: Adds a new element to the top of the stack. If the stack is bounded and full, this triggers a "Stack Overflow" condition. (O(1) time complexity)
Pop: Removes and returns the element currently at the top of the stack. If the stack is empty, this triggers a "Stack Underflow" condition. (O(1) time complexity)
Peek / Top: Returns the value of the top element without removing it from the stack. (O(1) time complexity)
IsEmpty: A helper function that returns a boolean indicating whether the stack contains any elements. (O(1) time complexity)
IsFull: A helper function (primarily used in fixed-size array implementations) that checks if the stack has reached its maximum allocated capacity. (O(1) time complexity)

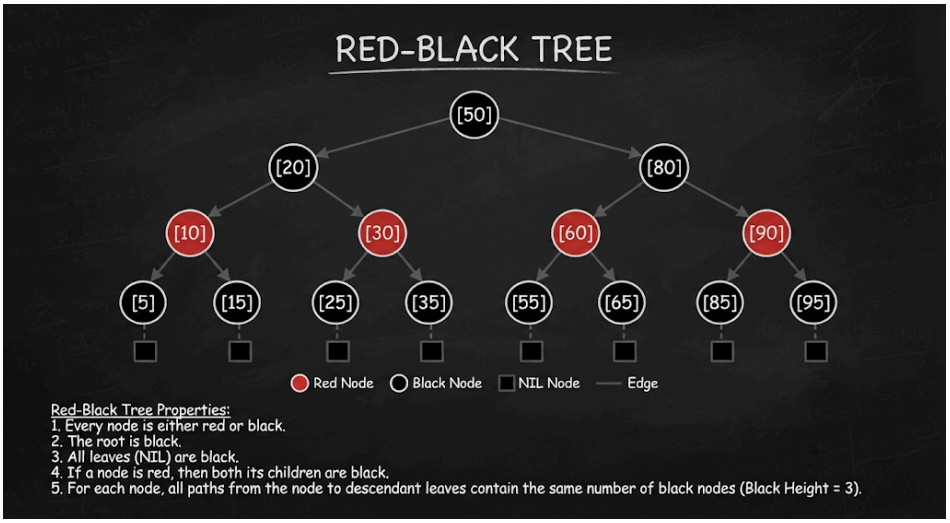
Hierarchical Structures:

A **hierarchical data structure** is an advanced way of organizing and storing information where elements are arranged in a multi-level, parent-child relationship. Unlike linear structures that form a single sequential line, hierarchical structures branch out across multiple tiers, making them ideal for representing data with natural organizational layers or nested relationships.

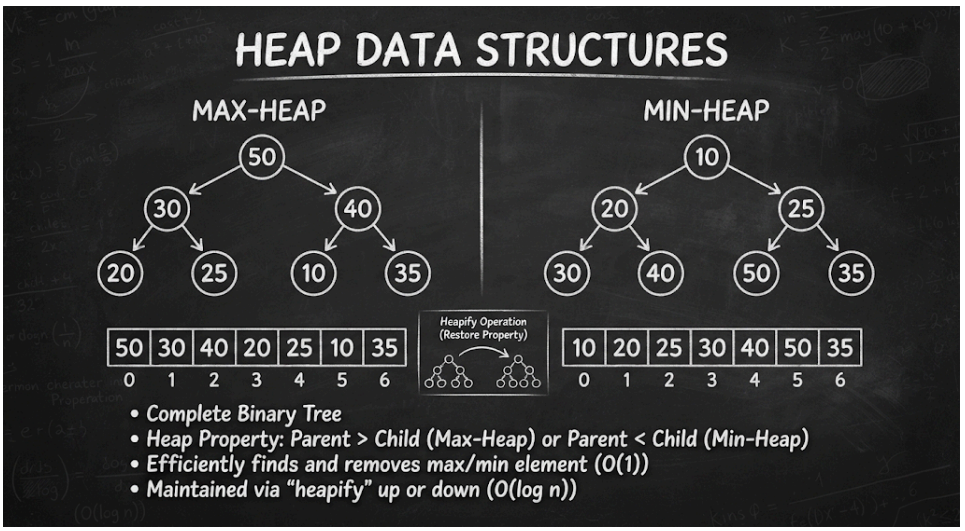


Binary Search Trees (BST): Ordered trees where each node maintains the invariant that its left subtree contains smaller values and its right subtree contains larger values.



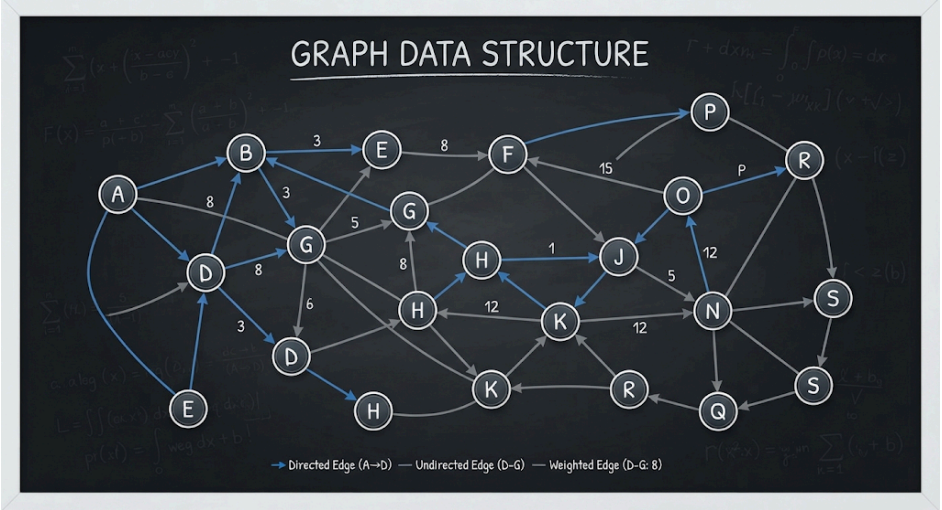


Red-Black Trees: Self-balancing binary search trees that utilize color-bit tracking to guarantee logarithmic search, insertion, and deletion times.



Heaps (Min/Max): Complete binary trees optimized for swiftly retrieving the highest or lowest priority element.

Network Data Structures:



A **graph (network data structure)** is a non-linear data structure used to model pairwise relationships between objects. It generalizes hierarchical structures by allowing arbitrary, complex connections rather than strict parent-child tiers.

Key Graph Variations

- **Directed vs. Undirected:** In a *directed graph*, edges have a specific direction (one-way street or dependency), whereas *undirected graphs* feature bidirectional, symmetric links.
- **Weighted vs. Unweighted:** *Weighted graphs* assign a cost, distance, or capacity value to each edge (crucial for routing algorithms), while *unweighted graphs* only track the presence or absence of a connection.

- **Cyclic vs. Acyclic:** Graphs may contain loops (cycles) that allow paths to loop back to a starting node, or they can be acyclic (like Directed Acyclic Graphs or DAGs) to enforce strict one-way operational dependencies.